Final Project (50 points)

This assignment covers all chapters to date throughout our Java Journey over the semester including:

- Strings
- Conditional Statements
- Loops
- Objects
- Arrays
- Inheritance, Polymorphism
- File I/O

You have come a long way and should be very proud of what you accomplished over the semester.

In this lab, you will be creating a license registration tracking system for the Country of Warner Brothers for the State of Looney Tunes. You will create four classes: Citizen, CarOwner, RegistrationMethods, and RegistrationDemo.

Citizen class

1. Create getter and setter methods for each of the instance vars, String firstName and String lastName (see UML below)
2. toString() returns a String with firstName, a space, and lastName (Note the csv file has these reversed)

Citizen

- firstName: String - lastName: String
+ Citizen() + Citizen(inFirst String, inLast String) +setFirstName(inFirst String): void +getFirstName(): String +setLastName(inLast String): void +getLastName(): String +toString(): String

Citizen has two constructors - a no arg constructor that sets firstName and lastName to "No Name" (ie firstName = "No Name") and one that sets firstName and lastName to values passed in.

CarOwner class ***extends*** Citizen and has two constructors. The no arg CarOwner constructor sets first and last name to "No Name", license to "Not Assigned", month to 0, and year to 0. Another constructor sets first and last name, license, month, and year to the values passed in.

CONTINUED ON NEXT PAGE

CarOwner

- license: String - month: int - year: int
+ CarOwner() + CarOwner(inFirst String, inLast String, inLicense String, inMonth int, inYear int) +setLicense(inLicense String): void +getLicense (): String +setmonth(inMonth int): void +getMonth(): int +setYear(inYear int): void +getYear(): int +toString(): String

CarOwner.java Method specifics

1) Create getter and setter methods for each of the instance vars String license, int month, and int year (see UML above)
2) Override .toString() and return a String with the following data: firstName, a space, lastName, then license, and then month/year. Since CarOwner extends Citizen, take advantage of super.toSting( ) for some of CarOwner's toString( ). Ensure that your output lines up nicely (see output.txt in Canvas) HINT: Remember what "\t" does.

RegistrationMethods class. There are two String instance vars, inputFileName and outputFileName. Also, two final static int constants, REG_MONTH set to 4 and REG_YEAR set to 2025 NOTE – RegistrationMethods requires importing java.util.Arrays and java.util.Scanner (or java.util.*) along with java.io.*.
Here is a summary of RegistrationMethods ():

1. setFileNames() – Prompt the user to provide the location of the csv file to be processed (registration.csv). The method should create a File object to test if a file exists at the path provided. If it does not, hold the user until the correct path is entered. Set inputFileName with this file/path. Prompt the user for file / path for the desired location of the output file (output.txt). Set outputFileName with this file/path. NOTE – Copy registration.csv to the folder with your java files. Or for Windows, you might want to see if you have a c:/tmp folder, if not create one and put registration.csv file in here (ie c:/tmp/registration.csv), save the output.txt file here (ie c:/tmp/output.txt). For Mac OS, you should have a /tmp folder already, but can create one if necessary and put registration.csv file in here (ie /tmp/registration.csv), save the output.txt file here (ie /tmp/output.txt).
2. getArraySize() – This method returns an integer based on the number of lines in the input file that has data. The best way to do this is to create a File object with inputFileName and then a Scanner object with the File object as an argument. You can then create an accumulator and use a while loop based on hasNextLine() that increments the accumulator. Before the while loop make sure to move the line pointer down to ignore the header line (ie input.nextLine()). You will also need to have input.nextLine() in the while loop block to move line to line otherwise you will have an infinite loop. Since working with Scanner with a File object argument, you will have a checked exception and either need to throws IOException (or optionally set up try and catch blocks).

3. processTextToArray(CarOwner[ ] inArray) – Adds new CarOwner objects to the CarOwner inArray passed in. For each line of the registration.csv file, a CarOwner object is added to inArray. Pitfall – Make sure you clear the first line of the csv (String input = inputStream.nextLine()). Then you can use split based on "," to create a String array. Once you split and have the components, how can you use these piece parts to create an object for inArray - think new, what constructor does new call and with what parameters? You will also need to create an int var (ie element) to put CarOwner objects into inArray elements. After adding each CarOwner object, you will need to increment element var. Since working with Scanner with a File object argument, you will have a checked exception and either need to throws IOException (or optionally set up try and catch blocks).
4. printArrayToFile(String inMsg, CarOwner[ ] inArray) - Prints inMsg and inArray to a file. Message Header is the inMsg passed in. HINT – You will need to use FileWriter set to append. Since working with PrintWriter with a FileWriter object argument, you will have a checked exception and either need to throws IOException (or optionally set up try and catch blocks).
5. flagOverdueOwners(CarOwner[] inArray) - generates and returns an array for vehicles whose registration have expired. This is defined as registrations that are over 12 months old based on current REG_MONTH and REG_YEAR. HINT - A helpful way to do this method and almostDue is how you completed findAll() in Lab7. Do one pass to get the number of occurrences. Create an array based on this size, and do another pass and add applicable owners to the new array.
6. flagAlmostDueOwners(CarOwner[] inArray) - Method that generates and returns an array for vehicles whose registration will expire in three months or less (months 10-12). The state of Looney Tunes sends a reminder three months out to the car owner.
7. getOutputFileName() – This method returns the string path of the output file so that users know where to get the output file.

RegDemo Class details

1. Create a RegistrationMethods object called `dmv`
2. Invoke setFileNames( ) using the object created above
3. Invoke getArraySize( ) and assign to a previously created int var (ie size).
4. Create a CarOwner[ ] called `ltState` set to size from above step.
5. Invoke processTextToArray( ) passing in `ltState` as an argument
6. Invoke printArrayToFile( ) passing the appropriate arguments (see RegistrationMethods methods summary above). For the String argument pass the following phrase "List of Car Owners"
7. Use flagOverdueOwners( ) to create a new CarOwner[ ] whose registration is over 1 yr old that is assigned to CarOwner[ ] `overdue` based on passing in `ltState` as an argument.
8. Use the appropriate RegistrationMethods method to print `overdue` with the String header "Owners with Expired Registration"

CONTINUED ON NEXT PAGE

9. Use flagAlmostDueOwners( ) to create a new CarOwner[ ] whose registration will expire within the next 3 months or less, but is not over one year old (registrations that are 10-12 months old) that is assigned to CarOwner[] `almostDue` based on the array `ltState`.
10. Use the appropriate RegistrationMethods method to print `almostDue` with the String header "Owners with registration expiring in three months or less"
11. Use getOutputFileName() to get the path of the output and then print out to the screen where the output file is located.

Rubric

Citizen.java – 5pts + 2pt javadocs
CarOwner.java – 10pts + 3pt javadocs
RegistrationMethods.java – 20pts + 4pts javadocs
RegistrationDemo.java – 4pts + 1pt javadocs

Submitting GitHub repo screenshot (1pt)

Submitting your work

For all labs you will need to provide a copy of all .java files. In addition to your .java files, you will need to provide output files of your console along with the output file (ie output.txt). The name of the console output file should match the class name and have the .txt extension such as TempProbOut.txt, ProChall3Output.txt. For GUIs such as JOptionPane, you will instead need to create screenshots. For Windows users, Snipping Tool is a great way to do this. Chromebook - Shift+Ctrl+Show Windows. Mac OS users, you can see how to take screenshots using the following url - https://support.apple.com/en-us/HT201361.